

SQL Dump Optimizer (db-dump-explore)

A set of tools for optimizing and working with standard database dump files (SQL dumps). The main goal is to significantly speed up the process of importing data into a database, especially for large dump files.

Problem

Standard tools like `mysqldump` or `pg_dump` often generate files where each `INSERT` statement adds only one or a few rows. With millions of records, importing such a file is very slow because each `INSERT` is a separate transaction and adds overhead on the database server side.

Solution

This script (`optimize_sql_dump.py`) processes a dump file and optimizes it in several ways to make the import much faster.

Main Features

- **Merging `INSERT` Statements:** The script combines many small `INSERT INTO ... VALUES (...), (...), ...` statements into a single, large statement, which drastically reduces the number of queries to the database.
- **Fast Load Mode (`--load-data`):** Generates `.tsv` files (tab-separated values) and a `.sql` file with `LOAD DATA INFILE` (for MySQL) or `COPY` (for PostgreSQL) statements. This is the fastest method for data import.
- **Split Mode (`--split`):** Splits a single large dump file into smaller `.sql` files, one for each table. This makes it easier to manage and import only selected tables.
- **Automatic Compression Detection:** The script can automatically read compressed files (`.gz`, `.bz2`, `.xz`, `.zip`), so you don't need to decompress them manually.
- **Support for MySQL and PostgreSQL:** Automatically detects the SQL dialect or allows you to specify it manually.

Requirements

- Python 3.x
- Optionally, the `tqdm` library for displaying a progress bar:

```
pip install tqdm
```

Usage

The script is operated from the command line.

```
python optimize_sql_dump.py [opcje] <file_input> [file_output]
```

Example 1: Basic Optimization (Merging INSERTs)

Processes `dump.sql.gz` and saves the optimized version to `dump_optimized.sql`.

```
python optimize_sql_dump.py --input dump.sql.gz --output dump_optimized.sql
```

Example 2: Split Mode by Table

Creates the `split_dump/` directory and places separate `.sql` files for each table from the `big_dump.sql` dump.

```
python optimize_sql_dump.py --input big_dump.sql --split ./split_dump/
```

Example 3: Fast Load Mode (Fastest Method)

Creates the `fast_load/` directory, containing:

- `.tsv` files with data for each table,
- `.sql` files with `LOAD DATA` (MySQL) or `COPY` (PostgreSQL) statements to load data from the `.tsv` files.

```
python optimize_sql_dump.py --input big_dump.sql --load-data ./fast_load/
```

Example 4: Working with a PostgreSQL Dump

If automatic detection fails, you can explicitly specify the database type.

```
python optimize_sql_dump.py --db-type postgres --input pg_dump.sql --output pg_dump_optimized.sql
```

Example 5: Optimizing a Single Table

Processes only the `CREATE` and `INSERT` statements for the `users` table.

```
python optimize_sql_dump.py --table users --input dump.sql --output users_only.sql
```

Example 6: Use the results of the program from the catalog `--load-data ./fast_load/`

```
parallel -j <n> "mysql -h <host> -u <user> <{}> " ::: *.sql
```

All Options

Option	Short	Description
--input	-i	Input dump file (can be compressed).
--output	-o	Output file for the optimized dump.
--db-type		Database type: mysql, postgres, or auto (default).
--table	-t	Optimize only the specified table.
--batch-size		Number of rows in a single merged INSERT statement (default: 1000).
--split [dir]		Splits the dump into separate files per table in the specified directory (defaults to current).
--load-data [dir]		Generates .tsv and .sql files for fast import (fastest option).
--verbose	-v	Displays additional diagnostic information and a progress bar.
--dry-run		Runs the script without writing any output files.